

Sorting & Extracting Unique Content

MODULE 2: FILE OPERATIONS, LINKS, AND DATA PROCESSING

INDEX

- Sort command
- Syntax of sort command
- Uniq command
- Core commands
- Why sort is necessary?

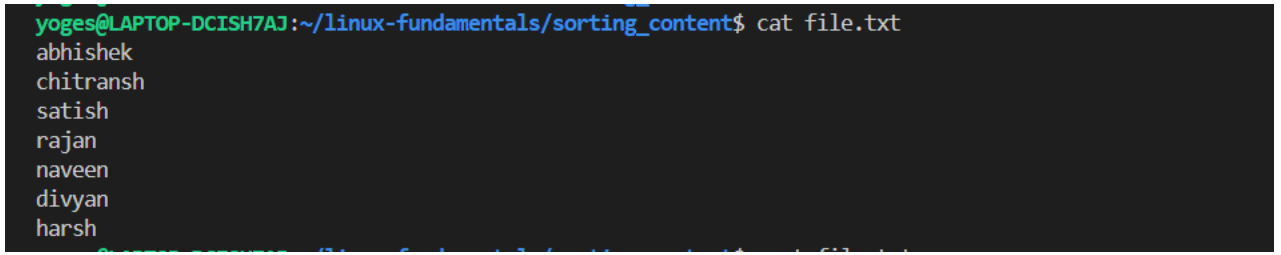
Sort command

The Linux **sort** command is a powerful command-line utility used to arrange the lines of text files in a specific order. By default, it sorts data alphabetically (lexicographically) in ascending order and prints the results to the standard output without modifying the original file.

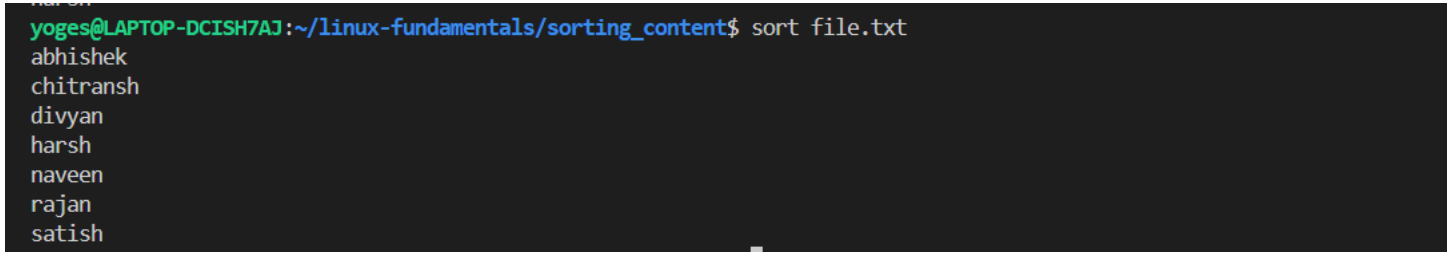
- Contents of a text files can be sorted **alphabetically**, **numerically**, by **month**, in **reverse**, and **removes duplicates.**
- Contents of a text file can be sorted, line by line.
- By default, the entire input is taken as the sort key.
  Blank space is the default field separator.

Sort the lines alphabetically

Let’s start with a simple example. Consider a file named ‘file.txt’ with the following content. Here we used cat command to display the content inside the file name ‘file.txt’.



To sort the lines alphabetically, you can use the following command: **sort file.txt**



Syntax of sort command

sort [OPTION] ... [FILE] ...

'options' refer to the various flags and parameters that can be used to customize the sorting behaviour, and 'file' is the name of the file to be sorted.

Common Options & Flags

Flag	Long Flag	Description
-n	--numeric-sort	Sorts strings by their numerical value instead of alphabetically.
-r	--reverse	Reverses the sorting order (e.g., Z to A or descending numbers).
-u	--unique	Removes duplicate lines, outputting only the unique values.
-k	--key=KEYDEF	Sorts based on a specific column or field index.
-t	--field-separator	Specifies a custom column delimiter (defaults to whitespace).
-f	--ignore-case	Folds lower-case to upper-case characters (case-insensitive sort).
-h	--human-numeric-sort	Sorts human-readable numbers (e.g., 2K, 5M, 1G).
-o	--output=FILE	Writes the sorted output directly to a file instead of the terminal.

Set Up Tabular Test Data

backup.tar	1	Jan	2048
notes.txt	3	Mar	512
images.zip	2	Feb	4096
scripts.sh	5	Apr	128

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ cat << 'EOF' > data.txt
> backup.tar  1  Jan  2048
> notes.txt   3  Mar  512
> images.zip  2  Feb  4096
> scripts.sh  5  Apr  128
> EOF
```

To practice advanced sorting options, we need a file that mimics a complex directory listing or data table. Let's create a file named data.txt containing filenames, links, sizes, and modification months.

Sorting Tabular Data by Field (**-k** Option)

By default, sort evaluates lines starting from the very first character.
The -k (Key) option lets you specify exactly which column to sort by.

Columns are separated by whitespace.

Command (Sort by Column 1 alphabetically): sort -k 1 data.txt

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort -k 1 data.txt
backup.tar  1  Jan  2048
images.zip  2  Feb  4096
notes.txt   3  Mar  512
scripts.sh  5  Apr  128
```

Sort based on File Size in Ascending Order (**-k** and **-n**)

File sizes are located in Column 4. Because they are numerical values, we must combine the field selector -k 4 with the -n (numerical) flag.

Command : sort -k 4 -n data.txt

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort -k 4 -n data.txt
scripts.sh  5  Apr  128
notes.txt   3  Mar  512
backup.tar  1  Jan  2048
images.zip  2  Feb  4096
```

Sort based on Month (**-M** Option)

Sorting months alphabetically (e.g., Apr, Feb, Jan, Mar) doesn't make chronological sense.

The -M (Month) flag tells Linux to parse three-letter abbreviations and order them according to the actual calendar year. The month values are in Column 3.

Command : sort -k 3 -M data.txt

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort -k 3 -M  data.txt
backup.tar  1  Jan  2048
images.zip  2  Feb  4096
notes.txt   3  Mar  512
scripts.sh  5  Apr  128
```

Sort based on Number of Links in Descending Order (**-r**)

The link count is located in Column 2. To sort numerical values in reverse order (highest to lowest), we combine the column selector -k 2, the numerical flag -n, and the -r (reverse) flag.

Command : sort -k 2 -nr data.txt

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort -k 2 -nr  data.txt
scripts.sh  5  Apr  128
notes.txt   3  Mar  512
images.zip  2  Feb  4096
backup.tar  1  Jan  2048
```

Every single sort operation worked exactly as expected:

sort -k 1 properly alphabetized the file names.
sort -k 4 -n handled the file sizes perfectly mathematically (128 up to 4096).
sort -k 3 -M successfully resolved the calendar timeline (Jan \(\rightarrow \) Feb \(\rightarrow \) Mar \(\rightarrow \) Apr).
sort -k 2 -nr accurately inverted the link count descending from 5 down to 1.

Uniq command

The **uniq** command in Linux is used to detect and remove duplicate lines from a file.

However, there is one critical rule you must remember before using it: uniq only detects duplicate lines that are adjacent (next to each other). Because of this, it is almost always paired with the sort command.

Core commands

What you want to do	Command to use	How it works
Remove adjacent duplicates	uniq shopping.txt	It filters out consecutive duplicates. *(Note: The first and last 'apple' will still show up because they aren't next to each other).*
Find ALL unique lines	sort shopping.txt | uniq	**This is the standard way.** Sorting brings all identical lines together so uniq can catch them all.
Show ONLY the unique lines	sort shopping.txt | uniq –u	The -u flag tells it to only print lines that appeared **exactly once** in the original file.
Show ONLY the duplicate lines	sort shopping.txt | uniq –d	The -d flag tells it to only print lines that had duplicates.
Count occurrences	sort shopping.txt | uniq –c	The -c flag prefixes each line with the **number of times** it appeared.

Why sort is necessary?

Run uniq without sorting first: **uniq sorting.txt**

Output: You will notice banana is grouped, but apple still appears twice because the two 'apple' lines weren't next to each other.

To get a clean, uniq list – pipe it through sort first: **sort shopping.txt | uniq**

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ uniq shopping.txt
apple
banana
orange
apple
banana
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort shopping.txt | uniq
apple
banana
orange
```

We get a perfect clean list of **apple, banana,** and **orange**.

Count and rank your data

See how many times each item appears, and sort it by the most frequent item:

sort shopping.txt | uniq -c | sort –nr

Output:

```
yoges@LAPTOP-DCISH7AJ:~/linux-fundamentals/sorting_content$ sort shopping.txt | uniq -c | sort -nr
      3 banana
      2 apple
      1 orange
```

By combining those three commands, we successfully transformed a messy list into a clean, ranked data report.